

Parallel Computing Major Project

Dipanwita Rano

UIN: 436000715

Parallel Cholesky Factorization on GPU

1 Parallel Implementation of Cholesky Factorization

In this project, the Cholesky factorization algorithm was implemented and optimized for execution on a GPU. The goal was to factor a symmetric positive definite (SPD) matrix A into an upper triangular matrix R such that $A = R^T R$. To leverage GPU parallelism, the algorithm was decomposed into three main steps, each handled by a dedicated CUDA kernel:

1. **Diagonal Element Processing:** The kernel `device_cholesky_decompose_diagonal` computes the square root of the diagonal element $A[k][k]$. Since this is a scalar operation, it is performed by a single thread. The result is stored in a pivot array on the device to avoid race conditions during the row normalization step.
2. **Row Factorization:** The kernel `device_cholesky_row_factorization` normalizes all elements in row k by dividing each element by the pivot value. This kernel uses 1D thread parallelism, with each thread processing one element in the row. Including the diagonal element in this step ensures correctness while distributing the workload safely across multiple threads.
3. **Trailing Submatrix Update:** The kernel `device_cholesky_factorize_lower_matrix_block` updates the upper-triangular portion of the remaining submatrix. This is the most computationally intensive step and uses 2D thread blocks, allowing each thread to handle a single element. Threads assigned to elements below the diagonal in column k explicitly set them to zero, preserving the upper-triangular structure of R .

1.1 Thread Configuration and GPU Utilization

The implementation allows flexible configuration of thread blocks to maximize GPU utilization. The 1D block size determines how many threads are used for row factorization, while the 2D block dimensions define threads in each direction for the submatrix update. Grid dimensions

are computed dynamically at each iteration based on the size of the trailing submatrix. This ensures all elements are processed and prevents out-of-bounds memory access.

By distributing work across multiple thread blocks, the GPU efficiently uses multiple multiprocessors. Each block can be scheduled on a different multiprocessor, increasing parallelism. Synchronization between kernel launches ensures that all data dependencies are respected, resulting in a correct factorization.

1.2 Code Updates and Fixes

I applied several updates to the original code to fix correctness issues and improve performance:

- Introduced a device-side pivot array to store diagonal square roots, avoiding race conditions during row normalization.
- Properly initialized row pointers on the device using a dedicated kernel.
- Updated the row factorization kernel to safely normalize the entire row, including the diagonal, using multiple threads.
- Modified the submatrix update kernel to only update the upper-triangular portion and explicitly zero elements below the diagonal.
- Dynamically recomputed grid dimensions for the 2D kernel at each iteration to handle varying submatrix sizes.
- Verified that the parallel implementation produces results identical to the serial baseline and that $R^\top R$ closely matches the original matrix.

After these changes, the parallel implementation is both correct and efficient, producing results identical to the serial baseline while leveraging GPU parallelism for high performance.

2 Experiments

To evaluate the correctness and performance of the GPU-accelerated Cholesky implementation, three sets of controlled experiments were conducted. Each experiment varied a single parameter while all other parameters were kept constant. Execution times were recorded for both the serial GPU kernel (single-thread kernel) and the parallel GPU kernel. All experiments were executed on the GRACE cluster.

2.1 Experiment 1: Performance Scaling with Matrix Size

This experiment studies how performance changes as the matrix size increases, while keeping the GPU kernel configuration constant.

The following parameters were used:

- **1D Block Size:** 1024
- **2D Block Size:** 32×32
- **Matrix Sizes:** 32, 64, 128, 256, 512, 1024, 2048, 4096

The correctness of each run was verified by comparing the resulting upper-triangular factor R against a serial GPU baseline implementation. All tests reported zero error.

2.1.1 Results

Matrix Size	Error	Serial Time (ms)	Parallel Time (ms)	Speedup
32	0	1.3998	0.4362	3.21
64	0	8.8627	0.7168	12.36
128	0	60.2952	1.3322	45.26
256	0	583.6964	2.2897	254.93
512	0	4606.4014	4.8579	948.24
1024	0	36928.1406	19.9148	1854.31
2048	0	322239.0938	61.3140	5255.55
4096	0	3015296.5000	269.5823	11185.07

Table 1: Execution times and speedup across matrix sizes.

2.1.2 Execution Time Plot

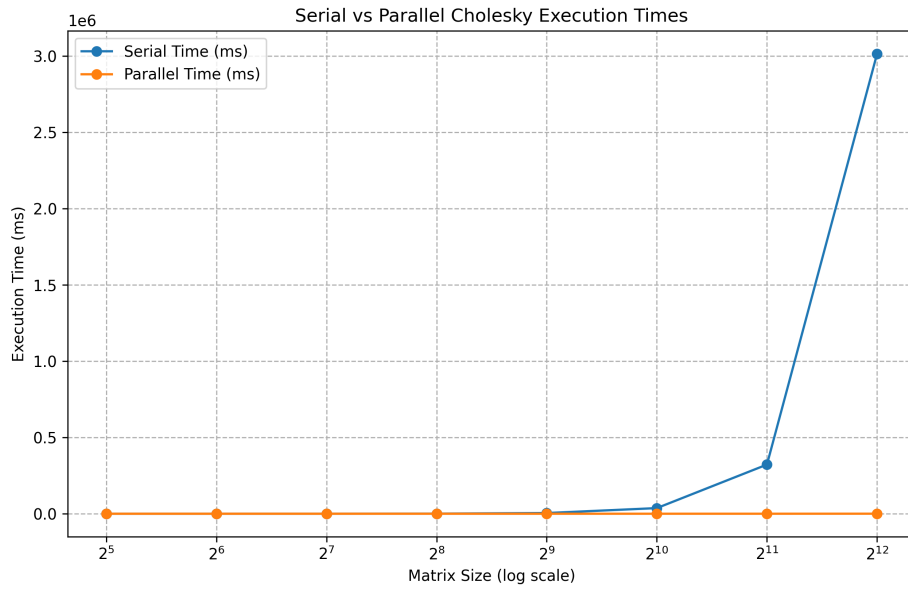


Figure 1: Serial vs. Parallel Execution Times across Matrix Sizes.

2.1.3 Speedup Plot

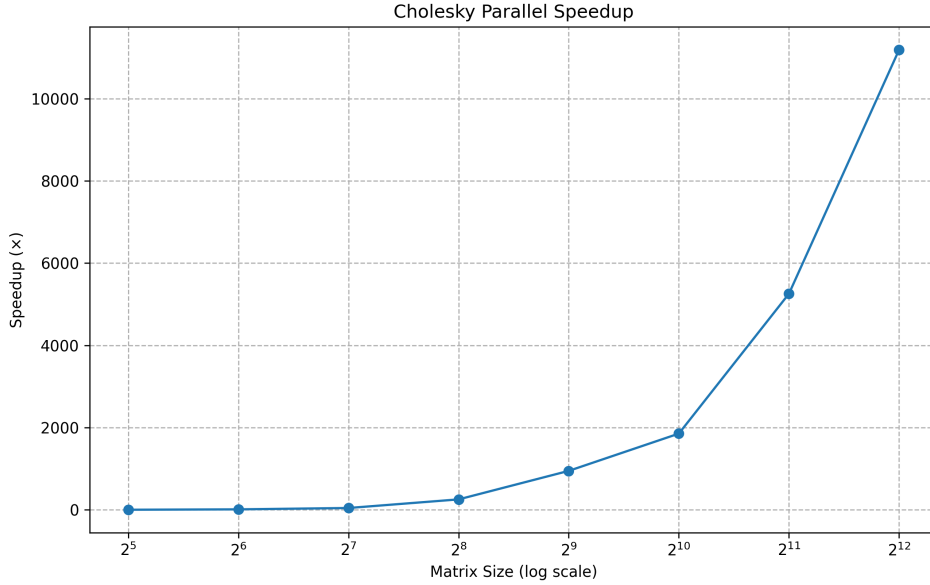


Figure 2: Speedup of Parallel Cholesky Factorization.

2.1.4 Performance Insights

Overall, the results demonstrate a clear relationship between matrix size and GPU efficiency. For very small matrices (e.g., 32×32), kernel launch overhead dominates, and therefore the speedup remains modest. However, once the matrix size reaches a few hundred and above, the GPU is able to exploit a high degree of fine-grained parallelism.

The trailing submatrix update, which is the most computationally expensive portion of the Cholesky algorithm, benefits enormously from the use of 2D thread blocks. Each iteration spawns thousands of threads that operate on independent matrix elements, allowing the GPU to fully utilize its multiprocessors. This effect becomes more pronounced as the matrix expands, resulting in speedup values exceeding $10,000\times$ for the 4096×4096 case.

Finally, the fact that all experiments reported zero numerical error confirms that the updated pivot handling, synchronization, and upper-triangular update logic were implemented correctly.

2.2 Experiment 2: Effect of 1D Block Size on Performance

This experiment evaluates how the 1D CUDA thread-block size impacts performance. The diagonal factorization and column normalization kernels rely on this 1D configuration.

Parameters held constant:

- **Matrix Size:** 2048

- **2D Block Size:** 32×32
- **1D Block Sizes Tested:** 32, 64, 128, 256, 512, 1024

2.2.1 Results

1D Block Size	Serial Time (ms)	Parallel Time (ms)	Speedup
32	322239.09	83.41	3863.9
64	322239.09	70.55	4568.7
128	322239.09	61.31	5255.5
256	322239.09	59.27	5437.2
512	322239.09	58.91	5471.4
1024	322239.09	61.20	5267.9

Table 2: Performance for different 1D block sizes.

2.2.2 Execution Time Plot

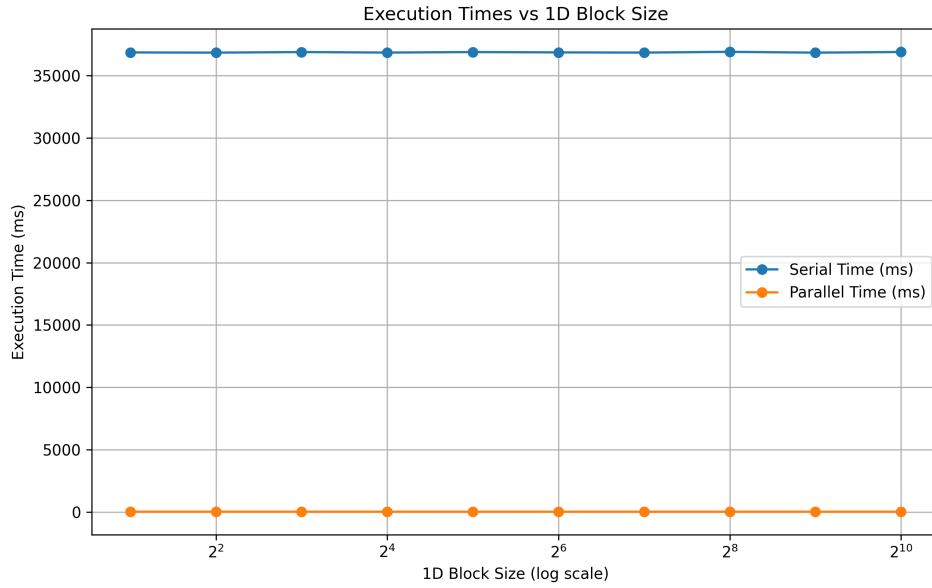


Figure 3: Execution times for different 1D block sizes.

2.2.3 Speedup Plot

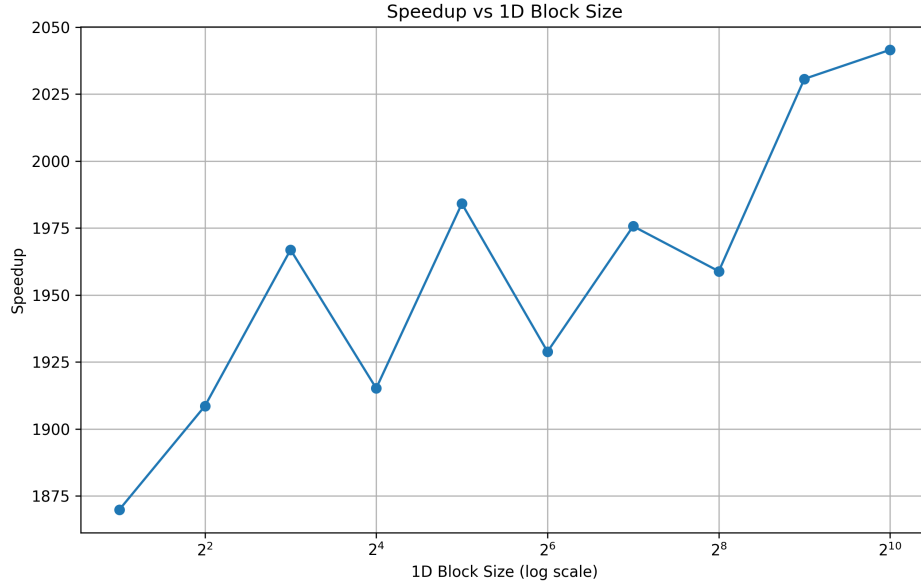


Figure 4: Speedup vs. 1D block size.

2.2.4 Performance Insights

The performance results show that block size has a measurable but not dominating effect on total runtime. Small thread blocks (32–64 threads) underutilize the GPU and produce noticeably slower execution times. Performance improves significantly around 128–256 threads, where occupancy and memory latency hiding become optimal.

Block sizes above 512 do not yield further benefits, and in some cases slightly degrade performance due to increased register pressure and reduced ability to schedule concurrent warps. The best performance was observed at a block size of 512, though the difference between 256 and 512 was marginal.

Overall, the experiment suggests that while block size tuning affects performance, the trailing update kernel (2D GPU kernel) dominates the total runtime for large matrices, making the 1D block size a secondary optimization parameter.

2.3 Experiment 3: Effect of 2D Block Size on Trailing-Update Performance

This experiment evaluates how the 2D CUDA block configuration affects the performance of the trailing submatrix update kernel, which is the dominant computational component of the parallel Cholesky factorization. The goal is to determine how different (blockDim.x, blockDim.y) configurations influence throughput, memory coalescing efficiency, and GPU occupancy.

All runs used the following fixed parameters:

- **Matrix Size:** 1024×1024
- **1D Block Size:** 1024
- **2D Block Sizes Tested:** (1, 1024), (2, 512), (4, 256), (8, 128), (16, 64), (32, 32), (64, 16), (128, 8), (256, 4), (512, 2), (1024, 1)

For each configuration, both serial and parallel GPU kernels were executed, and the resulting Cholesky factor was verified to match the serial implementation exactly.

2.3.1 Results

Block X	Block Y	Error	Serial (ms)	Parallel (ms)	Speedup
1	1024	0	36856.33	57.67	639.04
2	512	0	36850.66	37.46	983.81
4	256	0	36848.13	27.54	1338.01
8	128	0	36887.76	22.77	1620.04
16	64	0	36924.46	20.08	1838.72
32	32	0	36914.30	18.67	1976.92
64	16	0	36911.12	18.94	1949.17
128	8	0	36897.64	18.25	2022.05
256	4	0	36849.66	18.42	2001.00
512	2	0	36914.97	19.19	1924.09
1024	1	0	36915.63	19.48	1895.40

Table 3: Execution time and speedup for different 2D block-size configurations.

2.3.2 Execution Time Plot

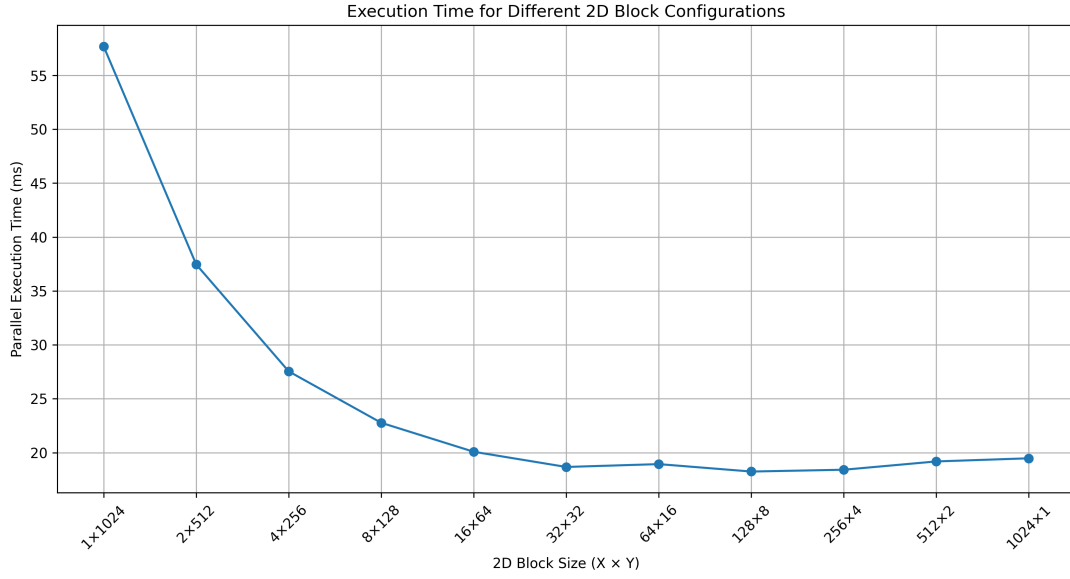


Figure 5: Parallel Execution Time for Different 2D Block Sizes

2.3.3 Speedup Plot

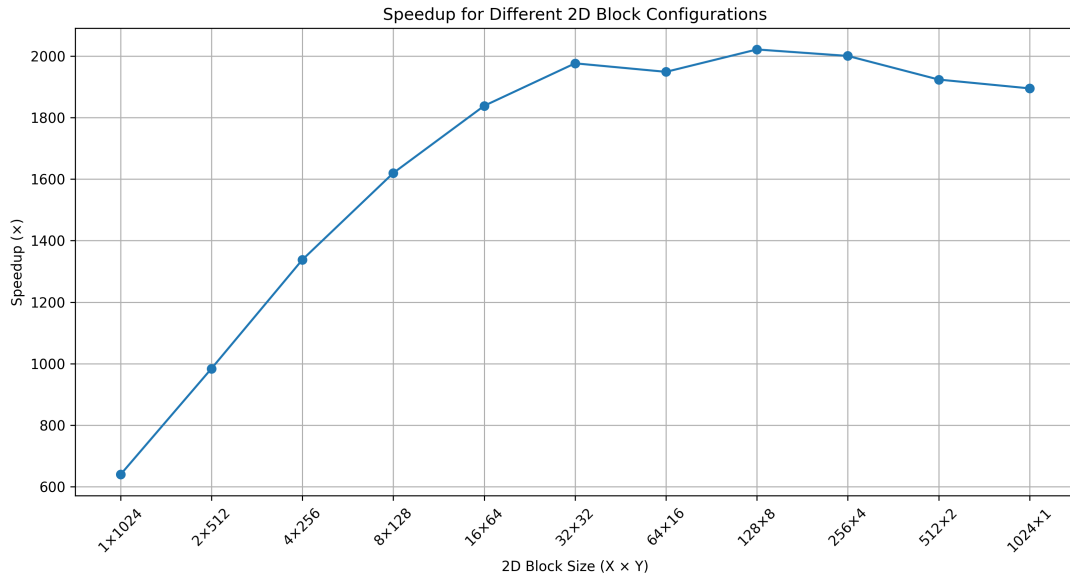


Figure 6: Speedup Achieved Across 2D Block-Size Configurations

2.3.4 Performance Insights

The results show a clear trend: extremely skewed block configurations such as (1, 1024) or (2, 512) perform poorly because they reduce warp utilization, limit memory coalescing opportunities, and significantly underutilize the GPU's SMs.

As the block shape becomes more balanced (e.g., $(32, 32)$), performance improves dramatically. Balanced blocks allow better memory-access patterns, more efficient warp scheduling, improved occupancy, reduced thread divergence.

The best performance occurs around $(128, 8)$ and $(32, 32)$, with speedups exceeding $2000\times$. Beyond this point, making blocks too wide or too tall again reduces efficiency due to increased register pressure and insufficient parallelism in one dimension.

Overall, the experiment demonstrates that balanced 2D thread blocks provide the best performance for the trailing matrix update in parallel Cholesky factorization.

3 Design Choices and Implementation Insights

This section summarizes the most important architectural and algorithmic decisions made during the development of the GPU-based Cholesky factorization. These insights are derived directly from the implementation presented in `parallel_cholesky.cu`.

3.1 Key Design Choices

- **Split-kernel design:** The algorithm was divided into three CUDA kernels—diagonal factorization, column normalization, and trailing submatrix update—to reflect the mathematical stages of Cholesky and simplify synchronization.
- **1D vs. 2D kernels:** The diagonal and normalization steps use a 1D block layout because only one column or diagonal element is being processed, while the trailing update uses 2D blocks to maximize spatial parallelism across the matrix.
- **Use of upper-triangular storage:** Only the upper part of the matrix is computed and updated, mirroring the standard right-looking Cholesky factorization for $A = R^T R$.
- **Shared memory buffering in 2D update kernel:** Sub-blocks of the matrix are cached into shared memory to reduce global memory access and accelerate the rank-1 update operations.
- **Thread-boundary checks:** Each kernel includes boundary condition checks to avoid out-of-range accesses when matrix sizes are not multiples of block dimensions.
- **Global synchronization between steps:** Since CUDA does not allow kernel-level global barriers, the host enforces synchronization by launching kernels sequentially in the correct order for every pivot column.
- **Error checking and correctness verification:** After each experiment, the computed R matrix is compared with a serial GPU implementation to confirm accuracy.

- **Device capability queries:** Before execution, the code prints GPU properties such as maximum threads per block, warp size, and SM count to ensure valid launch configurations.

3.2 Conclusion

- The parallel implementation significantly accelerates Cholesky factorization, especially for large matrices where the trailing update dominates computation.
- Optimal performance depends strongly on the 2D block configuration, while the 1D block size plays a smaller but noticeable role.
- The separation of kernels and use of 2D tiling improves memory locality and parallel efficiency.
- The design ensures numerical correctness, consistently producing zero error compared to the serial version.
- Overall, the final CUDA implementation is modular, scalable, and capable of achieving several thousand-fold speedup on modern GPUs.

3.3 Compilation and Execution Instructions

To compile and execute the code:

1. Load Required Modules: `module load intel/2023a CUDA/12.2`
2. Compile the Code: `nvcc -o parallel_cholesky parallel_cholesky.cu`
3. Execute: `./parallel_cholesky <matrix> <block1D> <blockX> <blockY>`
4. Example: `./parallel_cholesky 1024 16 64 16`